VS Code con Python · Guion del profesorado

UD2 · Cómo darlo, solucionario y rúbrica

Índice

1 Cómo repartir esto en clase **1**
- 1.1 La sesión A, minuto a minuto . . . 2
- 1.2 La sesión B: el depurador en el proyector . . . 2

2 Solucionario de la Fase 3 **3**
- 2.1 Cómo se detecta cada uno . . . 3
- 2.2 Respuestas de las demás fases . . . 4

3 Rúbrica analítica · P2.1 **5**
- 3.1 1 · Entorno montado y documentado (10 p) . . . 5
- 3.2 2 · Comprensión del error de tipos (10 p) . . . 5
- 3.3 3 · Manejo del depurador (25 p) . . . 5
- 3.4 4 · Identificación y corrección de los fallos lógicos (20 p) . . . 5
- 3.5 5 · Formateo y calidad del código (10 p) . . . 6
- 3.6 6 · Punto de ruptura condicional y punto de registro (10 p) . . . 6
- 3.7 7 · Memoria y reflexión (10 p) . . . 6
- 3.8 8 · Identidad y presentación (5 p) . . . 6

4 Montar la rúbrica en Moodle **7**

5 Material que hay que preparar antes **7**

1 Cómo repartir esto en clase

El documento del alumnado tiene tres partes y **no se dan seguidas**. Este es el reparto que funciona:

Sesión	Qué se hace	Partes	Duración
A (2 h)	Instalación, extensiones, primer programa, ejecutar	1.1 – 1.9	2 h
B (1 h)	El depurador, en el proyector y luego ellos	1.10	1 h
C (2 h)	Formateo, avisos, atajos + **Laboratorio de errores**	1.11 – 1.14 y Parte 3	2 h
D (2 h)	El ejercicio P2.1, en el aula	Parte 2	2 h + casa

Ojo

No des la Parte 3 antes que el depurador. El laboratorio de errores funciona porque ya saben poner un punto de ruptura: los seis primeros casos se resuelven leyendo el mensaje, pero los cinco del bloque B solo tienen gracia si pueden mirar dentro. Si lo das al revés, el bloque B se queda en «ah, vale» y se

olvida.

1.1 La sesión A, minuto a minuto

La instalación se come la clase entera si no la controlas. Orden que evita el caos:

Min.	Qué
0–10	Visual Studio Code **no** es Visual Studio. Y Python no se compila
10–35	Instalar Python **todos a la vez**, contando en voz alta la casilla del PATH
35–45	`python --version` en la terminal. **Nadie sigue hasta que a todos les responda**
45–60	VS Code y las cuatro extensiones
60–75	Crear la carpeta, abrirla, seleccionar intérprete
75–95	`hola.py` y las tres formas de ejecutarlo
95–120	`input()`, y provocar el TypeError a propósito

Idea clave

El truco de los 35–45 minutos. Que nadie avance hasta que a todo el mundo le funcione `python --version` parece que retrasa la clase, y es al revés: si dejas que unos sigan, te pasas los siguientes noventa minutos yendo de mesa en mesa arreglando instalaciones mientras la clase espera.
Encarga a los tres que terminen primero que ayuden a los demás. Funciona mejor que hacerlo tú solo y ellos aprenden explicándolo.

1.2 La sesión B: el depurador en el proyector

Media hora de demostración y media de práctica, en este orden:

1. Un programa con un bucle sencillo. Ejecutarlo normal: sale el resultado y ya está.
2. «¿Y si quiero saber cuánto vale `suma` en la tercera vuelta?» Alguien dirá «pon un print». Ponlo. Funciona. Ahora borra el print.
3. Punto de ruptura. F5. **Deja tres segundos de silencio** cuando se pare: el panel de Variables produce cierto efecto la primera vez.
4. `F10` cuatro o cinco veces enseñando cómo cambia `suma`.
5. **Cambiar un valor a mano** en el panel de Variables. Esto es lo que más les gusta.
6. `F11` entrando en una función, y `Mayús+F11` para salir.
7. El punto de ruptura condicional, con un bucle largo.

Idea clave

La frase que resume la sesión, y conviene decirla tal cual: **«un `print` te dice lo que tú habías pensado preguntar; el depurador te deja preguntar lo que no se te había ocurrido».**

2 Solucionario de la Fase 3

El programa tiene **tres fallos**, ninguno da error y la media correcta de `[7, 4, 5, 9]` es **6,25** (aprobado). El programa roto imprime **3.0** y dice SUSPENSO.

#	Línea	Qué está mal	Corrección
1	`suma = nota`	Sustituye en vez de acumular. Se queda con la última nota	`suma = suma + nota`
2	`media = suma / 3`	Divide entre 3 fijo, y hay 4 notas	`media = suma / len(lista)`
3	`if media > 5:`	Un 5 exacto sale suspenso	`if media >= 5:`

Versión corregida:

```
AUTOR = "Apellidos del alumno"


def calcular_media(lista):
    suma = 0
    for nota in lista:
        suma = suma + nota
    media = suma / len(lista)
    return media


def esta_aprobado(media):
    return media >= 5


notas = [7, 4, 5, 9]
media = calcular_media(notas)
print("Autor:", AUTOR)
print("Notas:", notas)
print("Media:", media)
if esta_aprobado(media):
    print("APROBADO")
else:
    print("SUSPENSO")
```

2.1 Cómo se detecta cada uno

#	Qué tienen que haber visto
1	En el panel de Variables, `suma` va 7 → 4 → 5 → 9 en vez de 7 → 11 → 16 → 25
2	Con 4 notas la división entre 3 no cuadra. Se ve poniendo `len(lista)` en Inspección
3	Solo aparece si prueban con `[5, 5, 5]`. La pista está en el enunciado

Ojo

El tercero es el importante y el que separa notas. Los dos primeros se ven en el panel de Variables casi solos. El tercero **no se manifiesta con los datos del enunciado**: con `[7, 4, 5, 9]` la media es

6,25 y da aprobado igual, esté el > o el >=.
Solo aparece probando con un caso límite. Eso es exactamente la idea de **valor límite** de la UD3, y merece la pena decirlo en clase: «acabáis de hacer, sin saberlo, lo que se llama una prueba de valores límite».
Quien encuentre los tres, y sobre todo quien explique **por qué** el tercero necesita otro juego de datos, tiene el nivel Excelente aunque falle otras cosas.

2.2 Respuestas de las demás fases

Fase 2. El error es `TypeError: can only concatenate str (not "int") to str`. Lo que hay que oír: `input()` devuelve texto, y Python no suma texto con número. Corrección: `int(input(...))`.

Fase 4. Cinco cambios de Ruff, cualquiera de estos vale: espacios alrededor de = y de los operadores · sin espacios dentro de los paréntesis · dos líneas en blanco entre funciones · el doble espacio de `def area_rectangulo` corregido · la sangría de 8 espacios pasada a 4 · espacio detrás de las comas.

La segunda pregunta es la buena: **el comportamiento no cambia**, y se comprueba ejecutándolo antes y después y viendo que sale lo mismo (28.27… y 20).

Fase 5. La nota imposible es el **13**, en la posición **7** de la lista. El punto de interrupción condicional `nota > 10` para una sola vez, con `contador = 7`.

Fase 6. Respuestas esperadas: `Ctrl`+F5 ejecuta sin parar en los puntos de ruptura, F5 sí para · F10 no entra en las funciones, F11 sí · el caso donde el `print` habría sido peor es la Fase 5, porque con un `print` dentro del bucle habrían salido 15 líneas y hay que buscar a mano · tres cosas que no hace el Bloc de notas: ejecutar, depurar, autocompletar, formatear, avisar de errores, renombrar con F2.

3 Rúbrica analítica · P2.1

Ocho criterios, cinco niveles. **Total: 100 puntos.**

3.1 1 · Entorno montado y documentado (10 p)

Nivel	Puntos	Descriptor
Excelente	10	Las tres capturas de la Fase 0 completas y legibles, con las cuatro extensiones y el intérprete visible en la barra de estado
Sobresaliente	8	Las tres capturas, falta algún detalle menor
Bueno	6	Dos de las tres capturas
Aceptable	4	Una captura, o capturas recortadas donde no se ve el contexto
Bajo	0	No documenta el entorno

3.2 2 · Comprensión del error de tipos (10 p)

Nivel	Puntos	Descriptor
Excelente	10	Explica con sus palabras que `input()` devuelve texto y que Python no mezcla tipos; corrige y comprueba
Sobresaliente	8	Explica la causa correctamente y corrige
Bueno	6	Corrige bien pero la explicación es vaga o copiada del mensaje
Aceptable	4	Captura el error pero no lo explica
Bajo	0	No hace la fase o no reproduce el error

3.3 3 · Manejo del depurador (25 p)

Nivel	Puntos	Descriptor
Excelente	25	Usa punto de ruptura, F10, F11, panel de Variables, pila de llamadas e Inspección, y las capturas demuestran cada uno en el momento adecuado
Sobresaliente	20	Usa todas las herramientas pedidas; alguna captura no demuestra bien el momento
Bueno	15	Punto de ruptura y Variables correctos; no usa Inspección o pila de llamadas
Aceptable	10	Pone puntos de ruptura pero las capturas no evidencian el uso real
Bajo	0	No usa el depurador: resuelve leyendo el código

3.4 4 · Identificación y corrección de los fallos lógicos (20 p)

Nivel	Puntos	Descriptor
Excelente	20	Los tres fallos, cada uno con línea, causa, evidencia del depurador y corrección; razona por qué el tercero exige otros datos
Sobresaliente	16	Los tres fallos documentados con los cuatro campos
Bueno	12	Dos fallos bien documentados
Aceptable	8	Un fallo, o los tres sin explicar cómo los encontró
Bajo	0	No encuentra ninguno, o entrega el programa corregido sin documentar nada

3.5 5 · Formateo y calidad del código (10 p)

Nivel	Puntos	Descriptor
Excelente	10	Antes y después, cinco cambios concretos identificados, y comprueba que el comportamiento no varía
Sobresaliente	8	Antes y después con cinco cambios; no comprueba el comportamiento
Bueno	6	Formatea y captura, pero identifica menos de cinco cambios
Aceptable	4	Formatea sin documentar
Bajo	0	No activa el formateo automático

3.6 6 · Punto de ruptura condicional y punto de registro (10 p)

Nivel	Puntos	Descriptor
Excelente	10	Condicional correcto que para una sola vez, y punto de registro con las variables interpoladas
Sobresaliente	8	Las dos herramientas funcionando, con alguna imprecisión
Bueno	6	Solo una de las dos
Aceptable	4	Encuentra el 13 pero con un punto de ruptura normal, parando quince veces
Bajo	0	No hace la fase

3.7 7 · Memoria y reflexión (10 p)

Nivel	Puntos	Descriptor
Excelente	10	Las cinco respuestas, con ejemplos sacados de su propia práctica
Sobresaliente	8	Las cinco respuestas correctas, genéricas
Bueno	6	Tres o cuatro respuestas
Aceptable	4	Responde de forma superficial o copiando la guía
Bajo	0	Sin memoria

3.8 8 · Identidad y presentación (5 p)

Nivel	Puntos	Descriptor
Excelente	5	Apellidos en los cinco ficheros, en la constante AUTOR y visibles en todas las capturas; PDF ordenado y capturas numeradas
Sobresaliente	4	Apellidos en los tres sitios, presentación mejorable
Bueno	3	Falta en alguno de los tres sitios
Aceptable	2	Solo en el nombre del PDF
Bajo	0	Sin identificar

Ojo

Los criterios 3 y 4 suman 45 de los 100 puntos, y es deliberado: son los que evalúan de verdad el uso del entorno, que es el RA2 del módulo. Lo demás es contexto.
Si alguien entrega los cinco ficheros funcionando perfectamente pero sin ninguna captura del depurador, **no llega al 5**, y hay que decírselo así de claro antes de empezar la tarea.

4 Montar la rúbrica en Moodle

Ojo

Moodle no importa rúbricas desde un fichero. No existe un formato de importación estándar como el XML de las preguntas: hay que crearla a mano una vez. La buena noticia es que después se puede **guardar como plantilla** y reutilizarla en cualquier otra tarea del centro.

Pasos:

1. Crea la tarea (*Añadir una actividad* → *Tarea*) y guárdala.
2. Dentro de la tarea: *Configuración* → *Calificación* → ***Método de calificación: Rúbrica***.
3. Guarda. Aparece un enlace nuevo: *Calificación avanzada* → ***Definir rúbrica***.
4. *Definir un nuevo formulario de calificación desde cero*.
5. Añade **ocho criterios** y, en cada uno, **cinco niveles** con los puntos y el texto de las tablas anteriores.

Marca estas casillas al final del formulario:

 Mostrar la descripción de la rúbrica durante la evaluación **Mostrar la descripción de la rúbrica a los estudiantes** ← la importante Mostrar puntuaciones por cada nivel al estudiante Permitir a los usuarios previsualizar la rúbrica

Idea clave

La segunda casilla hace que el alumnado **vea la rúbrica antes de entregar**, desde la propia página de la tarea. Es la diferencia entre una rúbrica que sirve para calificar y una que sirve para aprender: si saben cómo se corrige, apuntan ahí.

6. Cuando esté terminada: *Calificación avanzada* → ***Guardar la rúbrica como plantilla***. A partir de ahí, en cualquier tarea futura puedes elegir *Crear nuevo formulario a partir de una plantilla*.

Nota sobre la puntuación. Moodle suma los puntos de los niveles marcados y lo reescala a la calificación máxima de la tarea. Con esta rúbrica el máximo es 100, así que lo más cómodo es poner la **calificación máxima de la tarea en 10** y dejar que Moodle haga la conversión: un 100 sobre 100 se convierte en 10, un 45 en 4,5.

5 Material que hay que preparar antes

- ☐ Comprobar que los equipos del aula **permiten instalar extensiones** de VS Code. Si no, hablar con el coordinador TIC antes de la sesión A.
- ☐ Comprobar si Python ya está instalado en los equipos y con qué versión.
- ☐ Tener a mano un **pendrive con los instaladores** de Python y VS Code por si falla la red. Es lo que más veces salva la clase.
- ☐ Subir a Moodle los cinco ficheros `.py` de la Parte 2 ya escritos, para que no pierdan veinte minutos copiando a mano. **Con el `AUTOR` vacío**, para que lo rellenen ellos.
- ☐ Crear la tarea en Moodle con la rúbrica **antes** de mandar el ejercicio.
- ☐ Decidir si aceptas macOS y Linux en clase: los atajos cambian (`Cmd` en vez de `Ctrl`) y conviene anunciarlo.